

Introduction

Research Tools as Epistemic Infrastructure I

Scientific research operates through a layered ecology of tools, documents, and practices—each shaping what can be known, communicated, and verified. When these layers are fragmented, manuscripts, data, code, and figures can drift out of alignment, creating gaps that are structural rather than incidental. Survey and economic evidence documents the practical cost of failed reproduction attempts [baker2016reproducibility; freedman2015economics], but reproducibility is only the most visible face of a broader problem. Research software engineering, epistemic integrity, and the coordination between human researchers and AI collaborators all depend on whether the tools that produce research artifacts can themselves be transparent, testable, and self-documenting. Nosek et al. [nosek2018preregistration] argue that committing to analytical plans before data collection is a necessary structural reform; we extend this logic to the research build pipeline.

Research Tools as Epistemic Infrastructure II

One root cause is fragmentation—of attention (in the mind) as well as in the cyberphysical niche (documents, versions, and reproducibility artefacts). A typical research project scatters its artifacts across disconnected tools: LaTeX editors [lamport1994l^atex] for prose, Jupyter notebooks for analysis, ad-hoc shell scripts for figure generation, and manual copy-paste for integrating results into manuscripts. Each boundary between tools is a potential locus of desynchronization. The version of a figure embedded in a PDF may not match the code that ostensibly generated it, while tests may exercise analysis code without exercising the rendering path. Pimentel et al. [pimentel2019jupyter] studied a corpus described as 1.4 million Jupyter notebooks; in the valid execution subset, 24.11% ran without errors and 4.03% reproduced the recorded results. Those distinctions matter: execution success is not equivalent to result reproduction. Peng [peng2011reproducible] argues that reproducibility in computational science requires, at minimum, that the data and code underlying a published result be available for independent verification. Barba [barba2018terminologies] further documents how “reproducibility,” “replicability,” and

Related Work I

Gentleman and Temple Lang [@gentleman2007research] introduced the concept of a *research compendium*—a single unit of scholarly communication bundling code, data, and narrative. This vision has driven two decades of tooling, which can be grouped into four categories: workflow managers, literate programming systems, containerization approaches, and best-practice frameworks.

Related Work II

Workflow Managers

Snakemake [koster2012snakemake] uses a rule-based, Python-derived DSL to specify computational workflows as directed acyclic graphs of file-producing steps. It supports containerized execution and an extension architecture for execution and storage backends, while its central scope remains computational pipeline orchestration rather than this repository's combined manuscript, test, and document-provenance contract.

Nextflow [ditommaso2017nextflow] employs a dataflow programming paradigm with native support for container-based execution across heterogeneous computing environments (local, SLURM, AWS). Like Snakemake, Nextflow excels at bioinformatics pipeline parallelism but does not address manuscript production, document integrity, or the testing–publication coupling that characterizes research reproducibility.

CWL (Common Workflow Language) [amstutz2016cwl] provides a portable, YAML-based standard for describing computational workflows and their dependencies. Its strength lies in interoperability across execution engines (cwltool, Toil, Arvados)

template/: An Integrated Solution I

template/ was conceived as a structural antidote to this fragmentation. Rather than adding reproducibility as an afterthought—a Docker container wrapping an already-disjointed workflow [boettiger2015docker]—the template enforces integrity at the architectural level. It realizes Gentleman and Temple Lang's research compendium vision [gentleman2007research] at repository scale, bundling code, data, tests, manuscripts, and provenance into a single, pipeline-enforced system with version-controlled infrastructure [ram2013git]. It stands on four primary pillars:

template/: An Integrated Solution II

1. **Ergonomic Modularity:** A Two-Layer Architecture cleanly separates globally shared infrastructure (logging, rendering, validation, steganography) from project-specific logic (manuscripts, scripts, data). 28 infrastructure subdirectories (25 importable packages) comprising ~662 Python modules provide reusable services; projects consume them without modification.
2. **Execution Integrity:** Pipeline advancement is contingent on test passage. The configured source-coverage floors are 60% for infrastructure and 90% for each project. The static gate rejects prohibited mock-framework imports, while the suites exercise real filesystem and subprocess behavior and use local protocol servers or explicitly enabled live services for network boundaries. ~8,583 infrastructure tests and 4907+ project tests are discovered by the introspection build.

template/: An Integrated Solution III

3. **Automated Provenance:** Steganographic watermarking and cryptographic hashing are integrated directly into the rendering pipeline. Every generated PDF carries a SHA-256 fingerprint, an alpha-channel text overlay encoding the build timestamp and commit hash, and optionally a QR code linking to the repository. Provenance is not asserted by policy; it is enforced by architecture.

template/: An Integrated Solution IV

4. AI-Agent Collaboration and Skill-Based Agentic

Operations: A three-tier documentation architecture enables AI agents to operate at every level of the system. At the system level, `CLAUDE.md` asserts global architectural constraints. At the structural level, `AGENTS.md` files at every directory expose local API surfaces, file inventories, and integration contracts. At the module level, `SKILL.md` files—written to a discoverable YAML+Markdown schema aligned with the Model Context Protocol [[@anthropic2024mcp](#)]—define each infrastructure module as a reusable, self-describing tool. An agent invoking `infrastructure.rendering` does not need to read source code: it reads the `rendering/SKILL.md`, which declares the module's name, description, key imports, and example invocations in a machine-parseable YAML frontmatter block. This architecture is the practical realization of the skill-library paradigm established in the agent literature: Yao et al.'s ReAct framework [[@yao2023react](#)] demonstrated that interleaving reasoning traces with tool invocations dramatically improves

Scope and Contributions I

This paper is itself a product of the template it describes. The metrics populating its tables were computed by the introspection module documented in the Methods; the figures were rendered by the visualization code validated by the test suite described in Quality Assurance; the PDF carrying these words was assembled by the same YAML-declared pipeline whose architecture is the subject of the Results. This self-productive loop—where the system that is described is also the system that produces the description—is not incidental but structural, a concrete demonstration that `template/` can sustain the full lifecycle from source code to published artifact within a single, version-controlled, pipeline-enforced repository. Our contributions are:

- ▶ A formal description of the Two-Layer Architecture and Standalone Project Paradigm that enables N independent research projects to share infrastructure without coupling.

Scope and Contributions II

- ▶ A detailed specification of the 16-stage DAG in `infrastructure/core/pipeline/pipeline.yaml`: default full runs use 10 stages; `--core-only` runs 8; opt-in ebook, metadata, bundle, and archival stages via `--tags`.
- ▶ A comparative analysis positioning `template/` against nine peer tools—Snakemake, Nextflow, CWL, Quarto, Jupyter Book, R Markdown, DVC, Overleaf, OpenAI Prism—across fourteen feature dimensions, demonstrating that `template/` uniquely bundles the fourteen enforcement capabilities enumerated in §Results.

Scope and Contributions III

- An empirical evaluation across the canonical exemplar projects under projects/templates/ (templates/template_active_inference, templates/template_autopoiesis, templates/template_autoresearch_project, templates/template_autoscientists, templates/template_code_project, templates/template_data_descriptor, templates/template_eda_notebook, templates/template_formal, templates/template_gold_refinement, templates/template_literature_meta_analysis, templates/template_madlib, templates/template_methods_paper, templates/template_newspaper, templates/template_pitch_deck, templates/template_pools_rules_tools, templates/template_prose_project, templates/template_redacted_report, templates/template_registered_report, templates/template_search_project, templates/template_sia, templates/template_storybook, templates/template_template, templates/template_textbook)—including this meta manuscript from projects/templates/template_template which exercises introspection-derived metrics

Paper Organization I

The Methods describe the Two-Layer Architecture, Thin Orchestrator pattern, pipeline stages, and AI collaboration model. Results present quantitative metrics from multi-project execution, coverage analysis, and steganographic benchmarks. The Discussion addresses the Zero-Mock tradeoff, scalability implications, a detailed tool comparison, and future directions. The Infrastructure Module Reference provides detailed documentation for all 28 subdirectories. Security and Provenance describes the steganographic and cryptographic integrity layer. The Appendices provide pipeline, configuration, and comparative references.